

ViRGo — Beginner's Guide

A spoon-feeding onboarding guide. Read this once, top to bottom, and you can run and extend the project on your own. No prior graph or ML knowledge assumed.

1. What is this project?

ViRGo (*Virtual Role-Graph Embedding for Structural Identity*) is a research project aiming at a published paper. It **extends** an existing method called **Identity2Vec (I2V)** by Oluigbo et al.

Plain-English background. - A **graph** = dots joined by lines. Dots are **nodes** (e.g. web pages, papers, people); lines are **edges** (a link, a citation, a friendship). - Two nodes can share a **role** even if far apart — both *hubs* (many links) or both *bridges* (connect two groups). This role is a node's **structural identity**. - An **embedding** = a list of 64 numbers per node (its "fingerprint"). Nodes with similar roles should get similar fingerprints. - **I2V** builds these fingerprints with a *guided random walk* + a word model (Word2Vec). ViRGo studies a better way to feed structure into them.

2. Goal, contributions, methodology

Research question: which **virtual graph** (a graph that links nodes by *structural similarity*, not real edges) is best for a given dataset/task, and does a modern **GNN** over it beat I2V's walk method?

#	Contribution	Status
1	Virtual-graph study — build virtual graphs from I2V's Poisson/KL similarity; test on node classification, link prediction, anomaly detection; compare vs simpler graphs.	main work
Tech	GNN encoder (GraphSAGE/GIN/GAT) over the virtual graph, replacing walk+Word2Vec.	future
2	Embeddings as graph summaries for LLMs.	stretch, do not start yet

Pipeline (the method): graph → structural signal (degree + eigenvector centrality, cached) → KL-divergence λ → Poisson Ψ similarity → [virtual graph + GNN = future] → embedding → evaluation. Out of scope: hyperbolic/non-Euclidean space (a second paper).

3. Repository map (what each file does)

Core algorithm + run scripts live in the project root. The reusable CLI framework lives in `scripts/`. Everything else is data, docs, or results.

File / folder	Purpose
<code>identity2vec.py</code>	CORE baseline (frozen, never edit). The I2V Graph class + guided walk: uses node degree & eigenvector centrality → KL → Poisson Ψ to pick the next node.
<code>identity2vec_cached.py</code>	Same algorithm but caches the structural signals → identical output, ~200× faster . (Deliverable #1.)
<code>train.py</code>	The run file : read graph → make walks → Word2Vec (Skipgram) → save <code>.emb</code> . Flag <code>--cached</code> uses the fast path.
<code>make_labels.py</code>	Downloads & builds node labels (cora/citeseer from LINQS; webkb from the author's repo) and verifies they match our graph.
<code>prepare_linkpred.py</code>	Builds the link-prediction edge split (70/30) + fake "negative" pairs, leakage-free.
<code>eval_nodeclass.py</code>	Scores node classification → weighted F1.
<code>eval_linkpred.py</code>	Scores link prediction → AUC.
<code>plot_emb.py</code>	Draws an embedding as a 2-D picture (optional).
<code>scripts/ benchmark_config.py</code>	Single source of truth : dataset registry, hyperparameters, seed, split fractions.
<code>scripts/runner.py</code>	Glue: <code>embed()</code> , <code>run_linkpred()</code> , <code>run_nodeclass()</code> .
<code>scripts/results_io.py</code>	Saves a run to <code>results/NNN.<date>.<dataset>.<task>.csv</code> .
<code>scripts/utils.py</code>	Helpers: set seed, load embedding, next run id.
<code>scripts/main.py</code>	Terminal CLI (alternative to the notebook).
<code>notebooks/ reproduce_i2v.ipynb</code>	★ Primary entry point — click-through reproduction (see §5).
<code>input/</code>	Original graphs (<code>.edgelist</code>). Never edit .
<code>output/</code>	Trained embeddings (<code>.emb</code>).
<code>labels/</code>	Node categories for classification.
<code>splits/</code>	Edge splits for link prediction.
<code>results/</code>	Score sheets (<code>.csv</code>) + plots (<code>.png</code>).

File / folder	Purpose
docs/	The paper PDF, proposal, <code>notes.md</code> (lab notebook), this guide .
<code>CLAUDE.md</code>	The project's governing rules (read it).

4. Setup (one time)

1. The project uses the **conda environment i2v** (Python 3.12). Activate it: `conda activate i2v` — it already has `numpy 1.26.4`, `networkx`, `gensim 4.3.3`, `scipy 1.12.0`, `scikit-learn`, `pandas`, `jupyter`.
2. Open the notebook in VS Code (or `jupyter lab`) and pick the kernel **"Python (i2v)"**.
3. **Internet** is needed the first time you build `cora/citeseer` labels (a download).
4. A harmless `libtinfo.so` warning may appear in the terminal — ignore it.

5. How to run (the notebook = single entry point)

Open `notebooks/reproduce_i2v.ipynb`, run cells **top to bottom (Shift+Enter)**. You edit **one line**.

- **Step 1** — set `DATASET = "cora" | "citeseer" | "webkb_wisc"`. Every cell below follows it.
- **Steps 1.1–2** — build (if missing) and load the embedding `output/{DATASET}.emb`.
- **Step 3–5** — build/check labels → **node classification** (weighted F1) → save result.
- **Step 6 / 6.1 / 6.2** — **link prediction**: make the 70/30 split → retrain on the 70% → score AUC → save.

CLI alternative (terminal):

```
python scripts/main.py --list                # show datasets
python scripts/main.py --task nodeclass --dataset cora
python scripts/main.py --task linkpred --dataset cora --retrain
```

6. Datasets

Registered in `scripts/benchmark_config.py` (DATASETS). Labelled & ready: **cora** (7 classes), **webkb_wisc** (5 classes). **citeseer** edgelist exists but labels may fail an overlap safety-check (it was derived). Others (politics, enzymes, ...) are listed for future use.

7. Evaluation — splits, settings, metrics

The most important idea: the two tasks split differently.

Node classification — *transductive*. Build the embedding on the **whole** graph once, then split the **nodes** 80/20 (stratified, keeps class ratios), train a logistic-regression classifier on 80%, score the held-out 20%. The test *labels* are hidden → no leakage. **Metric: weighted F1.**

Link prediction — *inductive, leakage-free*. Split the **edges** 70/30. A *spanning tree* stays in train so the graph stays connected; equal numbers of fake "negative" (non-edge) pairs are added. **Retrain a fresh embedding on the 70% train graph only** (so test edges are never seen). Turn each node-pair into an edge feature with the **Hadamard** product, train logistic regression, score AUC on the held-out 30%. **Metric: AUC**.

Fixed settings: `seed = 42`; `embedding dimensions = 64`, `num_walks = 10`, `window = 10`, `epochs = 1`; `walk_length = 40` in the repo/CLI default, **80** in the notebook config and the paper (a recorded deviation — both are valid, just be consistent). Classifier = logistic regression (the paper's L-BFGS / L2). No separate validation set (we don't tune).

Caveat (research rigor): results come from a **single split**. Small datasets are noisy across seeds — for a paper, report **mean $\pm$ std over ≥ 5 seeds** and add macro-F1.

8. Results & outputs

Where	What
<code>output/{ds}.emb</code>	full-graph embedding (node classification)
<code>output/{ds}_lp.emb</code>	train-only embedding (link prediction)
<code>splits/{ds}_train.edgelist</code> , <code>_train_neg.txt</code> , <code>_test_pos.txt</code> , <code>_test_neg.txt</code>	link-pred split
<code>results/NNN.DD.MM.{ds}.{task}.csv</code>	one run: a #META JSON header (all settings) + metric rows; auto-numbered

Reproduced so far: cora node classification **weighted F1 = 0.6992** ($\approx$ paper $\rightarrow$ reproduced). cora link prediction **AUC $\approx$ 0.80** (paper 0.84, close). webkb_wisc: F1 $\approx$ 0.31 ($\approx$ random — see §10) and link AUC = 0.63 (webkb is **not** a paper benchmark). Cache fix: **207 $\times$ faster, byte-identical**.

9. Reproducibility rules (non-negotiable)

- **seed = 42 everywhere** (split, init, sampling). Keep `--workers 1` (gensim is otherwise non-deterministic).
- **Never modify anything in `input/`** — write derived files alongside or into `output/`.
- A result counts as "**reproduced**" **only within ± 0.03** of the paper.
- **Log every run and decision in `docs/notes.md`** (the lab notebook).

10. Coding rules & where to change things

Coding style (match I2V): `argparse` CLI; `build_graph()` / `learn_embeddings()` / `main(args)` shape; **fewest functions**, each short with a one-line docstring; self-explanatory file names; models expose `train(epochs)` not `fit()`; add no dependency without need.

✅ **Safe modification zones** - Pick dataset/task → notebook **Step 1** (DATASET), then run the node-class or link-pred cells. - Tune hyperparameters → notebook **Step 1** vars, or `scripts/benchmark_config.py` (I2V_PARAMS) for the CLI. - Add a dataset → drop the `.edgelist` in `input/`, add an entry to `DATASETS`, add labels in `labels/`. - Add a task → new `eval_<task>.py` + register it in `scripts/runner.py` (TASKS).

❌ **Do NOT touch** - `identity2vec.py` (frozen baseline) and don't move it or `train.py`. - Files in `input/`. The seed. The location of `CLAUDE.md` (root).

11. Known issues / gotchas

- **Notebook clobber:** if the `.ipynb` is open in an editor while a script edits it, the editor can overwrite changes. Close it before bulk edits; reload from disk after.
- **citeseer labels** may be rejected by the overlap safety-check (derived edgelist) → node classification can stop. Link prediction still works.
- **webkb structure ≠ content:** I2V learns *structural roles*; webkb classes are *content topics* and the graph is heterophilous, so node-class F1 ≈ random. Expected, not a bug. Use `webkb_wisc` for *link prediction*.
- **Single-split variance** on small graphs (§7 caveat).
- The original I2V recomputes centrality inside the walk loop (very slow) — always use `--cached`.

12. Deliverables & how to continue

Deliverable	State	Next action
1. Cached I2V (identical + faster)	✅ done	—
2. <code>virtual_graph.py</code> (top-K Poisson/KL builder)	🕒 next	build it: connect each node to its K most structurally-similar nodes
3. GNN encoder (GraphSAGE/GIN/GAT) over the virtual graph	🕒	add torch + torch-geometric
4. Anomaly detection eval (AUC/AP)	🕒	start standalone on GADBench/PyGOD
5. Benchmark table + ablation (vs I2V, over graph & K)	🕒	extend <code>results/</code>
6. Paper draft	🕒	—

Where to start as the new intern: (1) run the notebook on **cora** end-to-end and confirm F1 ≈ 0.6992; (2) read `identity2vec.py` and `train.py` to see how walks become embeddings; (3) skim `docs/notes.md` for history; (4) then begin Deliverable #2 (`virtual_graph.py`).

13. Mini-glossary

Node/edge — dot / line. **Embedding** — 64-number fingerprint per node. **Structural identity** — a node's role (hub, bridge), independent of position. **Transductive** — embed the whole graph, hide only labels. **Inductive** — retrain on a sub-graph so test items are unseen. **Weighted F1** — accuracy-like score that accounts for class sizes. **AUC** — probability the model ranks a real edge above a fake one. **Leakage** — letting test information into training (forbidden).